

CSE 482 Project - Part III: Summarization

A. Conclusion

This project focuses on building a movie recommendation system using the MovieLens latest-small dataset. The work completed so far included collecting and preprocessing the data, storing the processed tables in a MySQL database running through Docker, and comparing user-based and item-based collaborative filtering methods.

The main goal was to predict how a user may rate a movie they have not rated yet. The project used the `userId`, `movieId`, and `rating` fields as the main features for collaborative filtering. Movie genre information was also prepared and analyzed because it gives additional context about the types of movies being recommended.

The best performing model was the item-based collaborative filtering model. With $K = 100$ nearest neighbors, item-based collaborative filtering achieved an RMSE of 0.9016 and an MAE of 0.6801. This performed better than both the baseline user-mean model and the user-based collaborative filtering model.

The baseline model produced an RMSE of 0.9413 and an MAE of 0.7347. The user-based collaborative filtering model produced an RMSE of 0.9676 and an MAE of 0.7476, which means it did not outperform the baseline. This result makes sense because the user-item matrix is highly sparse. The matrix density was only about 0.017, meaning most users rated only a very small fraction of the total movies. Because of this, user-to-user similarity can be weak or noisy.

Item-based collaborative filtering performed better because item relationships are more stable in this dataset. Popular movies receive ratings from many users, which makes movie-to-movie similarity more reliable than user-to-user similarity. This is why the item-based model is the better choice for the recommendation part of the project and for the web interface.

The most important features were the user rating behavior and the movie identifiers. Ratings are the core signal because they directly represent user preferences. Movie IDs are needed to build the item-based similarity model. Genre information was useful for understanding the movie catalog and for explaining recommendations, but the main prediction models were still driven by rating patterns.

Training efficiency was mainly affected by three factors: dataset sparsity, number of users and movies, and the value of K . Increasing K from 50 to 100 improved both collaborative filtering models, especially the item-based model. However, larger K values also increase the amount of computation required because more neighbors must be considered for each prediction.

B. Future Work

There are several ways the project could be improved in future work.

- First, the recommendation system could be improved by using a hybrid approach that combines collaborative filtering with genre-based information. This would help when there are not enough ratings for a user or movie.
- Second, matrix factorization methods such as SVD could be tested. These methods are commonly used in recommendation systems and may handle sparse rating matrices better than nearest-neighbor methods.
- Third, the model could be improved by tuning more parameters, including the number of neighbors, similarity metric, and minimum number of ratings required for candidate movies.
- Fourth, time information could be used more directly. In the current work, timestamps were converted to readable dates, but the model did not account for changes in user preferences over time.
- Fifth, the web interface could be improved by allowing users to select genres, filter recommendations, and compare recommendations from the user-based and item-based models.

For the remaining project work, the item-based collaborative filtering model will be used as the main recommendation engine because it gave the best performance in the analysis. The web interface will use the database and model outputs to display rated movies, recommend unrated movies, and show similar users.

C. Report

Overall, the analysis showed that item-based collaborative filtering is the strongest model for this dataset. The main reason is that the MovieLens data is very sparse, so finding reliable similar users is harder than finding similar movies. The results also showed that model performance can improve by tuning the number of nearest neighbors.

The project is now ready to move into the web interface stage. The previous steps already produced the key pieces needed for the application: cleaned data stored in MySQL, user rating history, item-based recommendations, and similar-user outputs. These components can be connected to a simple web application that allows a user to log in, view rated movies, receive recommendations, and see similar users.

The final project report will combine the previous sections into one complete document, including the problem definition, data collection and preprocessing, model analysis and results, conclusions, future work, and lessons learned.